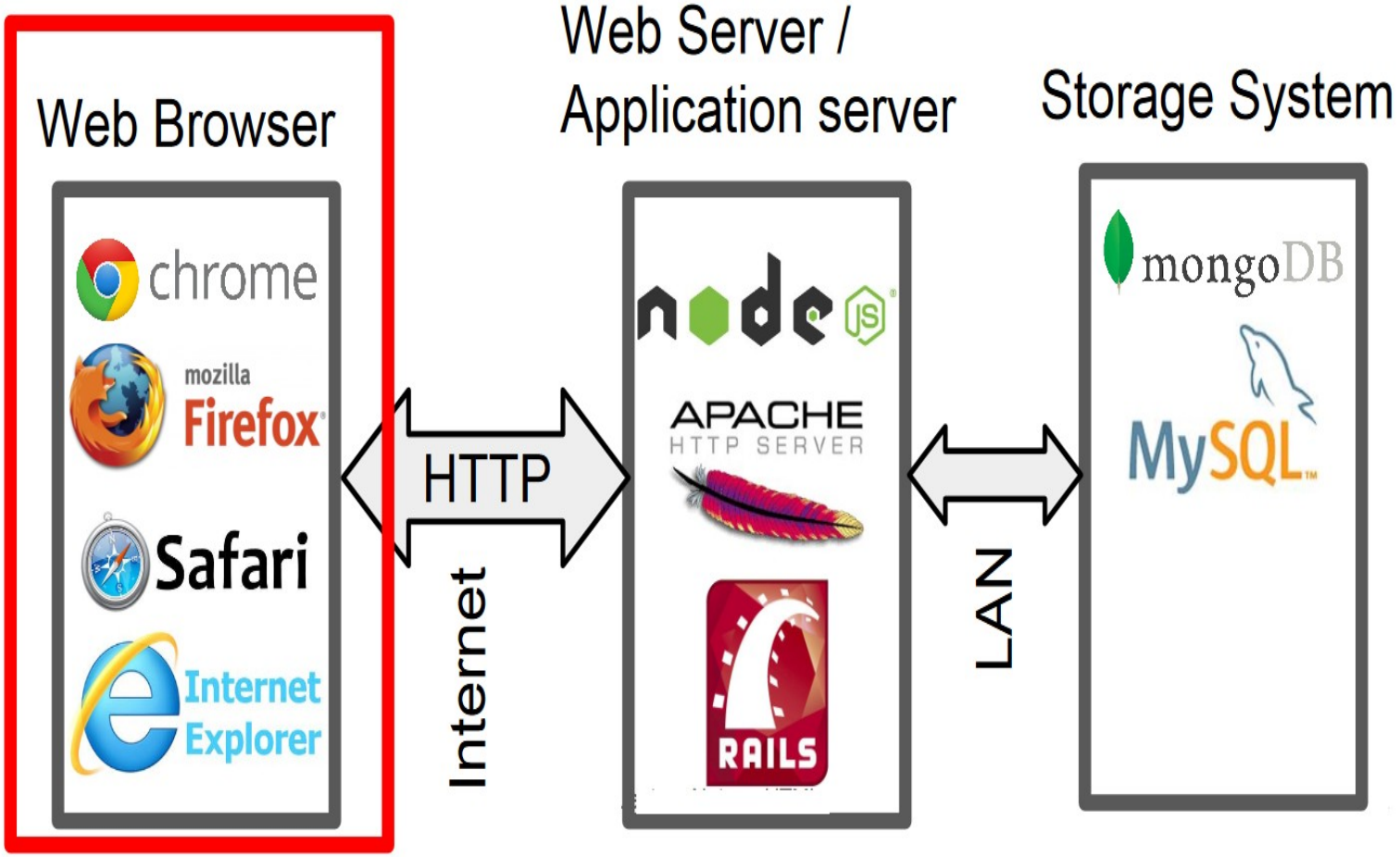
Web and Database Security

Web application architecture



Uniform Resource Locators (URLs)

- The process of visiting a website begins with the browser determining the IP address of the ***web server*** that is hosting – DNS lookup

  http://www.example.com/Dir1/file.html

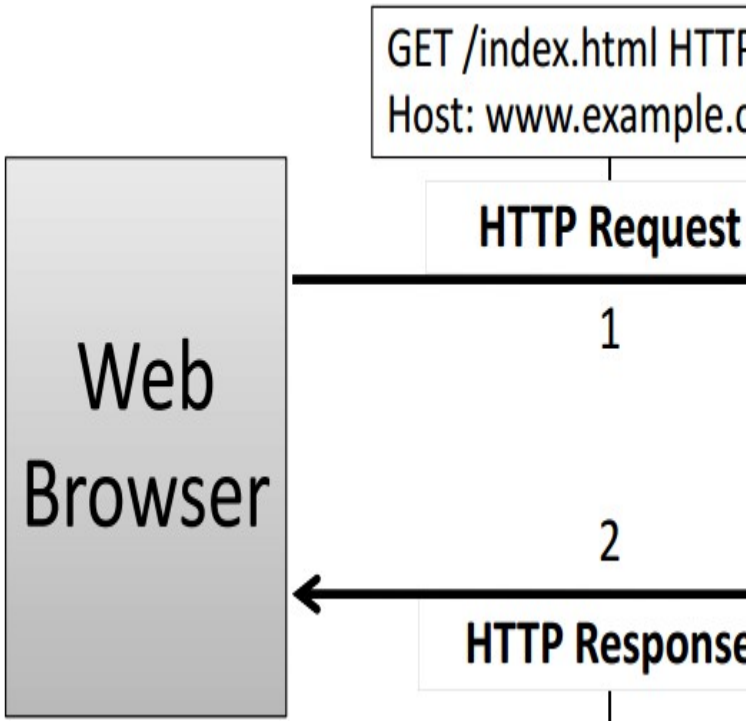
- A web browser identifies a web site with a uniform resource locator, or URL.

Connecting to a Web Server

- After the IP address of the web server is resolved through DNS, the client makes a TCP connection to a specified port on the web server, which is, by default, port 80 for HTTP.

- Other protocols besides HTTP could also be used in a URL
  - Port: 21 File Transfer Protocol (FTP)
    Port: 80 Hypertext Transfer Protocol (HTTP)
    Port: 443 Hypertext Transfer Protocol over TLS/SSL (HTTPS)

HTTP Request




```
HTTP/1.1 200 OK
Server: Apache/2.2.3 (CentOS)
Last-Modified: Fri, 30 Jul 2010 15:30:18 GMT
ETag: "573c1-254-48c9c87349680"
Accept-Ranges: bytes
Content-Type: text/html; charset=UTF-8
Connection: Keep-Alive
Date: Sun, 15 Aug 2010 15:24:11 GMT
Age: 4341
Content-Length: 596

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<HTML>
<HEAD>
 <META http-equiv="Content-Type" content="text/html; charset=utf-8">
 <TITLE>Example Web Page</TITLE>
</HEAD>
<body>
<p>You have reached this web page by typing &quot;example.com&quot;,
 &quot;example.net&quot;,&quot;example.org&quot; or &quot;example.edu&quot;
 into your web browser.</p>
<p>These domain names are reserved for use in documentation and are not available
 for registration. See <a href="http://www.rfc-editor.org/rfc/rfc2606.txt">RFC 2606</a>,
 Section 3.</p>
</BODY>
</HTML>
```

Lack of Confidentiality in HTTP

- By default, HTTP requests and responses are delivered via TCP over port 80.
- No encryption
- If an attacker could intercept the packets being sent between a web site and a web browser, he would gain full access to any information
- Could result in a ***man-in-the-middle***

HTTPS AND CERTIFICATE

Plaintext communication

- Attacker may have access to network communication between browser and server
- Passive attacks
  - Eavesdrop on network traffic
- Active attacks
  - Inject network packets
  - Modify packets
  - Reorder, replay packets
  - Block packets

HTTPS

- To solve the confidentiality problem inherent in HTTP
  - ***HTTPS (HTTP over secure socket layer)***

- Incorporates an additional layer of security known as ***SSL* (*secure socket layer*),** or a newer implementation, known as ***TLS* (*transport layer security*).**

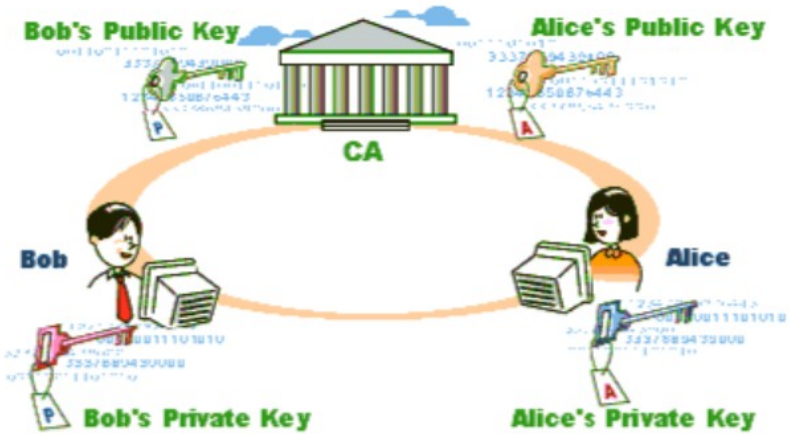
- SSL and TLS rely on the notion of a ***certificate*** to verify the identity of the server and establish an encrypted communication channel between the web browser and the web server.

- Only one way: SSL/TLS does not allow the server to verify browser identity

Web Server certificate

- **Certificate authority:** well-known, trusted server that certifies public keys.
- **Certificate:** a document encrypted with the secret key of a certificate authority
  - Identifies a particular service along with its public key

- **Trust:** Browser trusts certification authority
- Browser can validate the certificate came from the certification authority and see who the certification authority thinks the browser is talking to



Attacks on Web browser

- As Web browser is an integral part of the way people use computers, they are the popular targets for attacks
- Different kinds of attacks:
  - Session hijacking
  - Phishing
  - URL obfuscation
  - Privacy attacks
  - Cross Site Scripting (XSS)
  - Cross Site Request Forgery (CSRF)

Phishing

- Basic idea:
  - Get users to visit an evil Web site
  - Convince them that the evil Web site is actually a legitimate site (such as bank)
  - Trick the user into disclosing personal information (password, credit card number, etc.)
  - Use the personal information for evil purposes such as identity theft.
  - Hides its activity from the user, either by redirecting them to the real site or presenting a notice that the site is "*down for maintenance*."
- How to attract users?



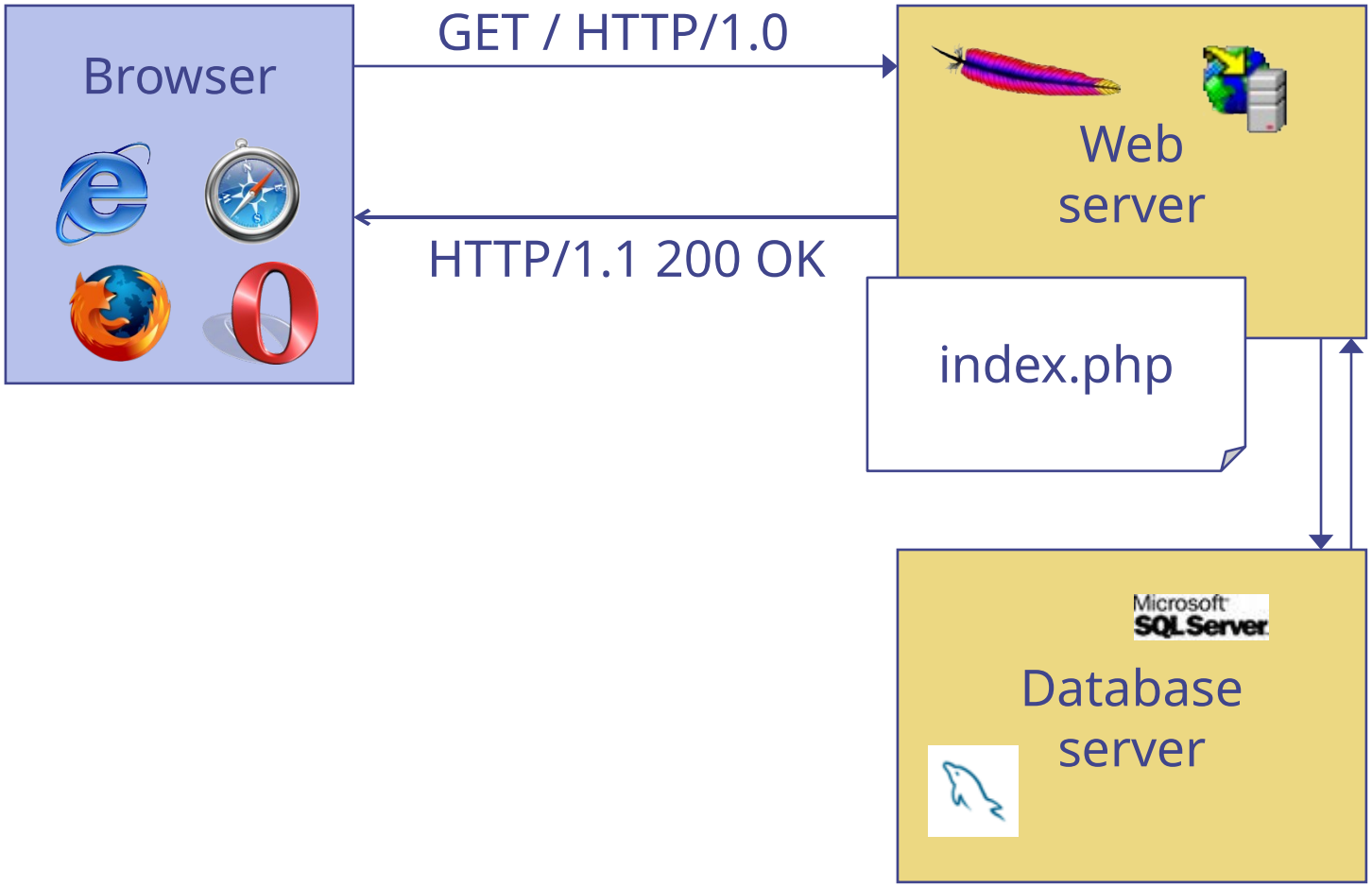
Common Web vulnerabilities

Sans Top 10

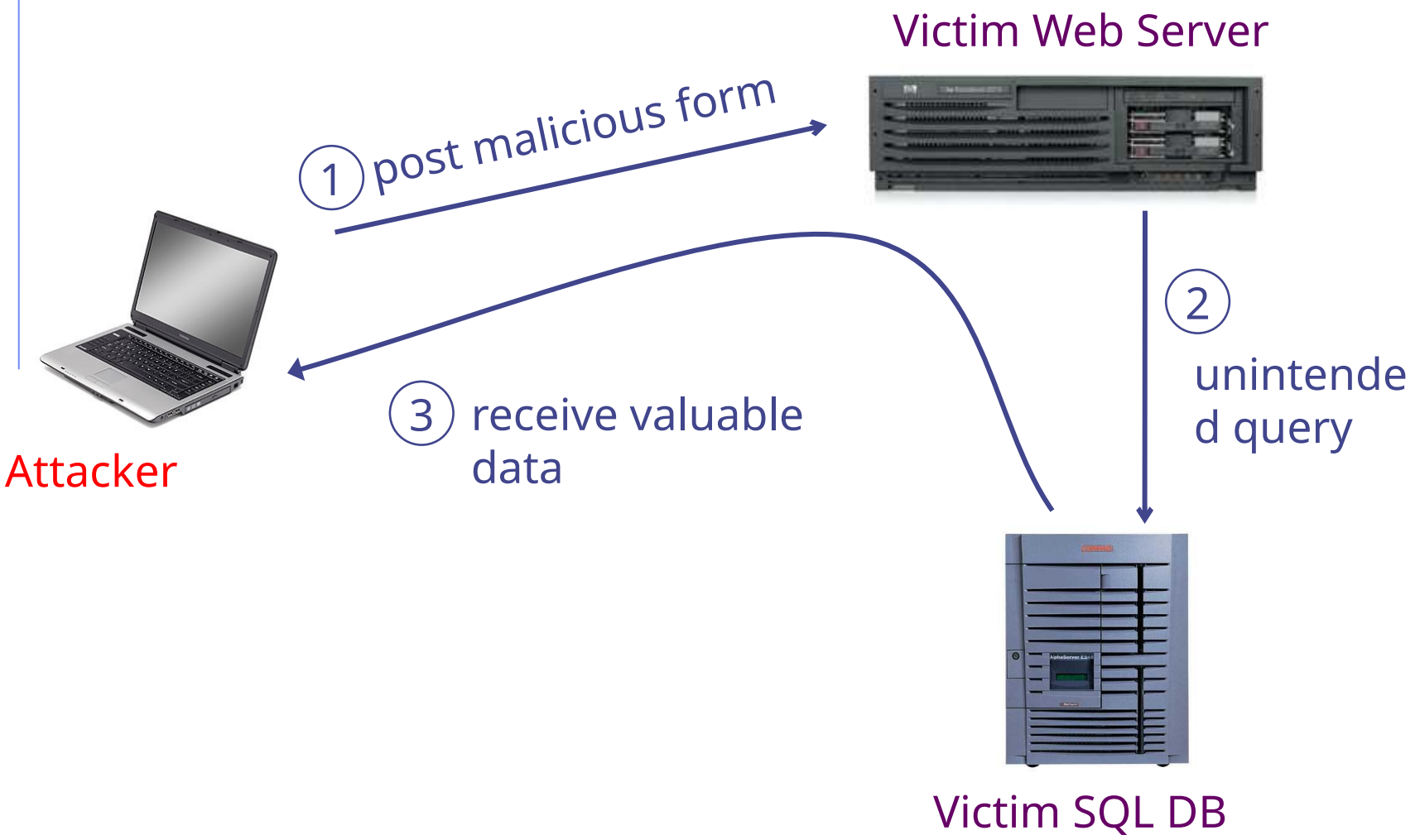
- SQL Injection
  - Browser sends malicious input to server
  - Bad input checking leads to malicious SQL query
- XSS – Cross-site scripting
  - Bad web site sends innocent victim a script that steals information from an honest web site
- CSRF – Cross-site request forgery
  - Bad web site sends request to good web site, using credentials of an innocent victim who “visits” site
- Other problems
  - HTTP response splitting, site redirects, …

SQL Injection

Dynamic Web Application



Basic picture: SQL Injection



What is SQL Injection?

- Input Validation Vulnerability
  - untrusted user input in SQL query to back-end database
  - *without sanitizing the data*
- Specific case of more general *command injection*
  - inserting untrusted input into a query or command
- Why Bad?
  - supplied data can be misinterpreted as a command
  - could alter the intended effect of command or query

SQL Injection Attacks

- SQL injection is the placement of malicious code in SQL statements, via web page input.
- https://www.w3schools.com/sql/sql_injection.asp

UserId: 105 OR 1=1

```
SELECT * FROM Users WHERE UserId = 105 OR 1=1;
```

```
SELECT UserId, Name, Password FROM Users WHERE UserId = 105 or 1=1;
```

More SQL Injection examples

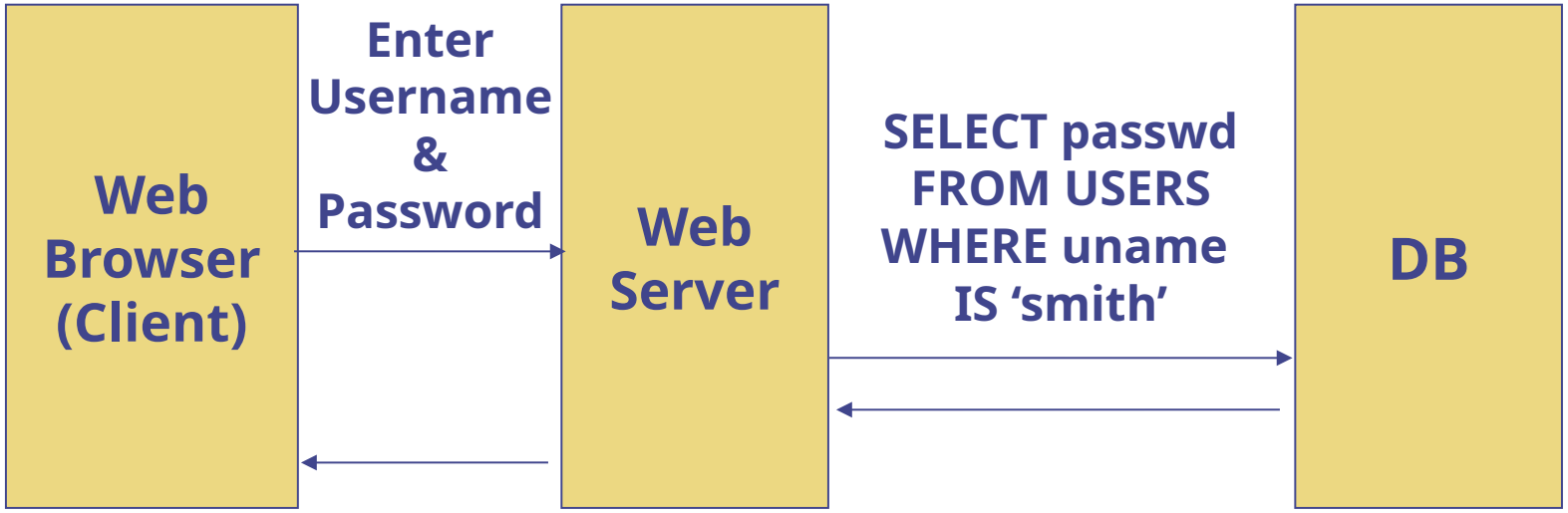
User id: 105; DROP TABLE Suppliers

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

Normal SQL Query



Normal SQL Query



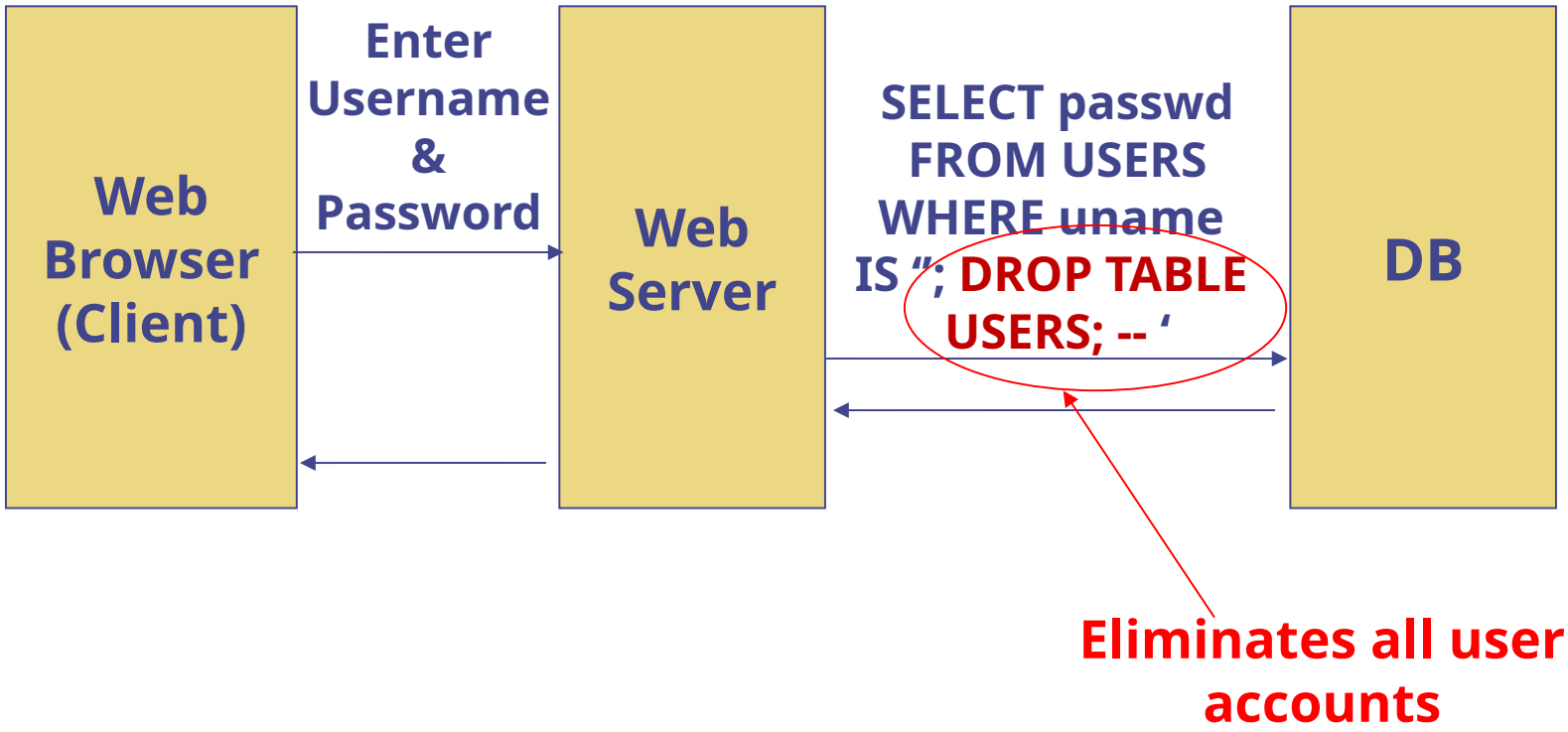
Normal Query

SQL Injection Examples



SQL Injection Examples

Malicious Query



More SQL Attacks

- Create new users:

  ‘; INSERT INTO USERS (‘uname’,’passwd’, ‘salt’)
  VALUES (‘hacker’,’38a74f’, 3234);

- Password reset:

  ‘; UPDATE USERS SET email=hcker@root.org
  WHERE email=victim@yahoo.com

Preventing SQL Injection

- Input validation
  - Filter
    - Apostrophes, semicolons, percent symbols, hyphens, underscores, …
    - Any character that has special meanings
  - Check the data type (e.g., make sure it's an integer)

- Whitelisting
  - Blacklisting chars doesn't work
    - forget to filter out some characters
    - could prevent valid input (e.g. username O'Brien)
  - Allow only well-defined set of safe values
    - Set implicitly defined through regular expressions

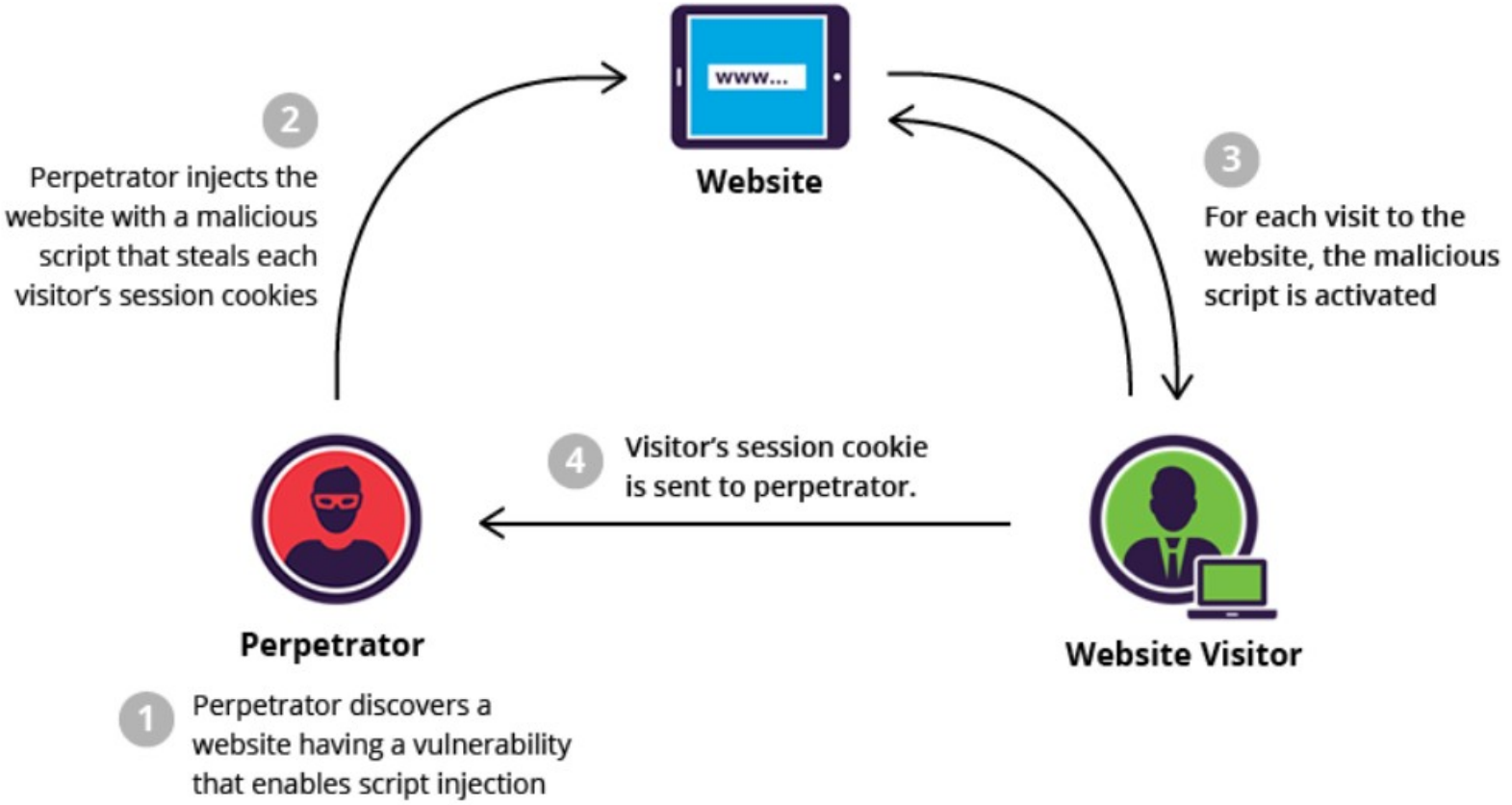
Cross Site Scripting (XSS)

Reading materials:

https://blog.hubspot.com/website/cross-site-scripting

Cross Site scripting attack

To successfully execute a XSS attack, a perpetrator has to **locate a vulnerability** in a web application and then inject malicious script into its server



Cross Site Scripting Vulnerability

- A web application is vulnerable to cross site scripting attacks if it:
  - stores user input without sanitizing, validating, or encoding it
  - dynamically includes user data in its HTTP responses without sanitizing or validating it

Consequences: XSS attack

- User accounts can be compromised
- Trojan horse programs activated
- page content modified
- Misleading users into surrendering their private info
- Finally, session cookies could be revealed, enabling a perpetrator to impersonate valid users and abuse their private accounts.

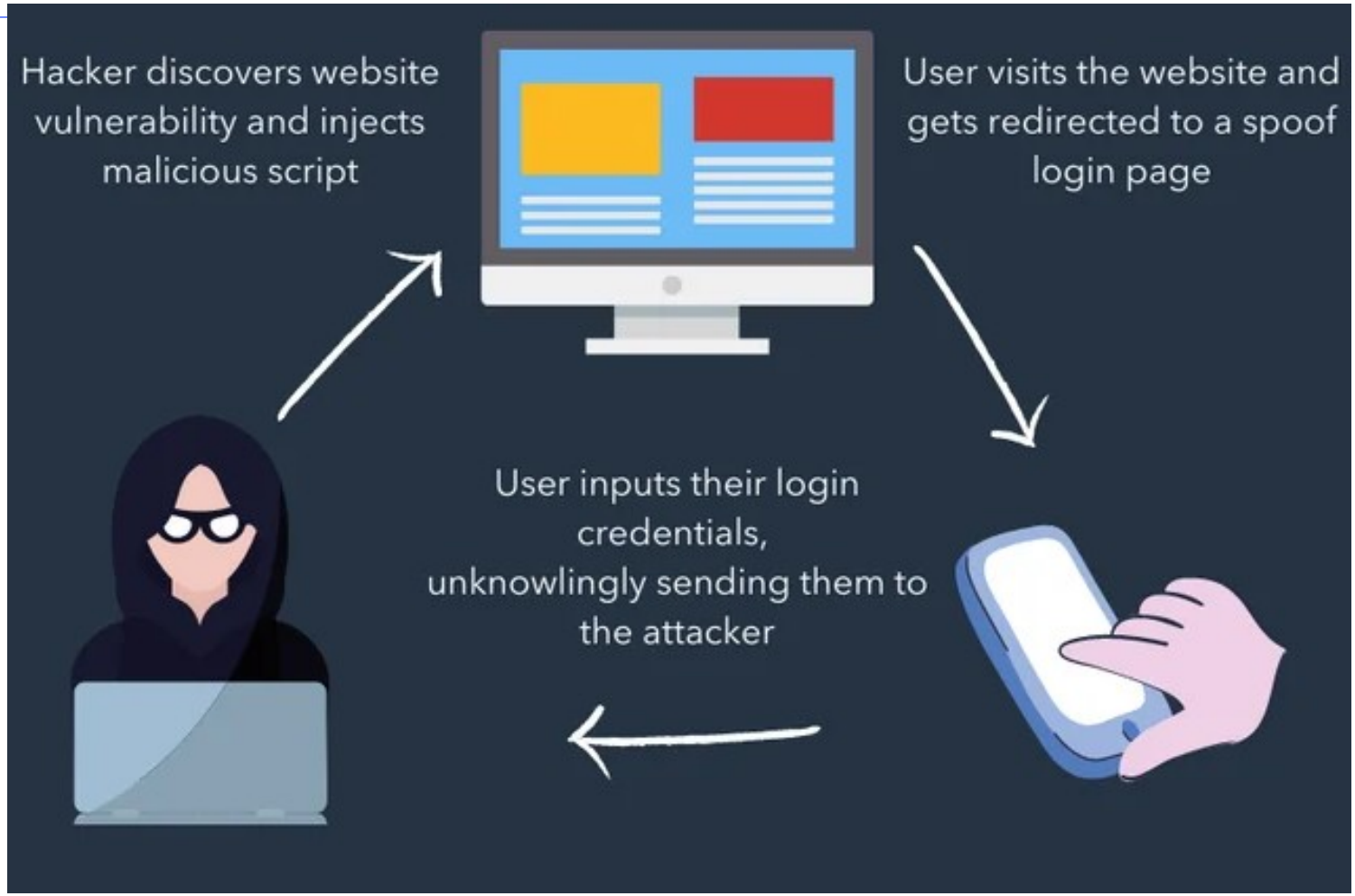
- Two types:
  - Stored XSS and
  - Reflected XSS

Stored vs. Reflected XSS

- Stored XSS (or persistent XSS) is the more damaging of the two. It occurs when a malicious script is **injected directly into a vulnerable web application**.

- Reflected XSS involves the reflecting of a malicious script off of a web application, onto a user's browser.
  - The script is embedded into a link, and is only activated once that link is clicked on.

Stored XSS example



XSS attack on eBay

- https://www.bbc.com/news/technology-29241563

- EBay has been compromised so that people who clicked on some of its links were automatically diverted to a site designed to steal their credentials.

Cross Site Request Forgery (CSRF)

https://www.imperva.com/learn/application-security/csrf-cross-site-request-forgery/

CSRF

- It is an attack vector that tricks a web browser into executing an **unwanted action** in an application to which a user is logged in.

CSRF Eaxmple

- A typical GET request for a $100 bank transfer might look like:

  GET http://netbank.com/transfer.do?acct=PersonB&amount=$100 HTTP/1.1

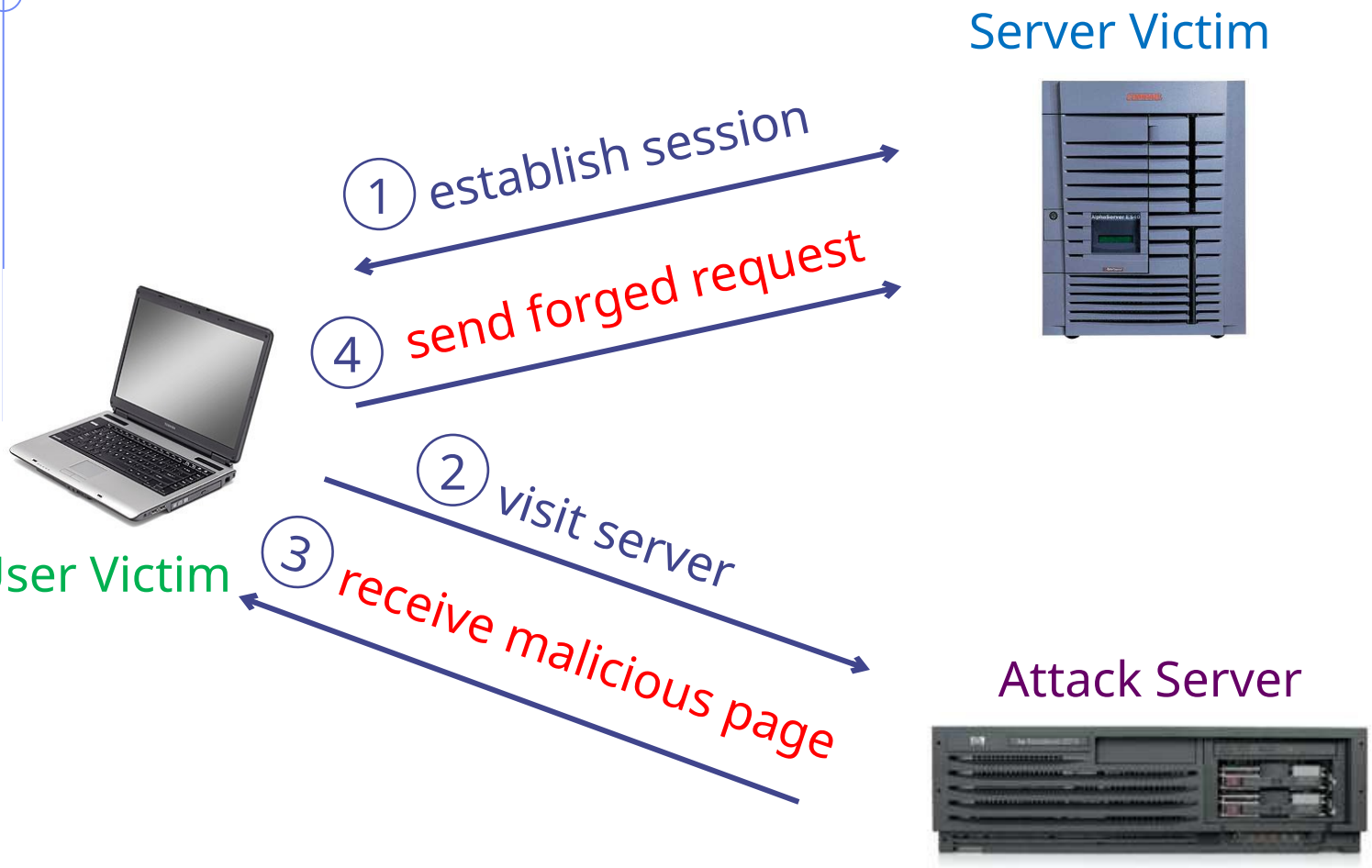
- A hacker can modify this script so it results in a $100 transfer to their own account.

  Get http://netbank.com/transfer.do?acct=AttackerA&amount=$100 HTTP/1.1

- A bad actor can embed the request into an innocent looking hyperlink:

  Read more!

Basic picture



Q: how long do you stay logged on to Gmail?

CSRF Mitigation

Best practices include:

- ✓ Logging off web applications when not in use
- ✓ Securing usernames and passwords
- ✓ Not allowing browsers to remember passwords
- ✓ Avoiding simultaneously browsing while logged into an application

CSRF Recommendations

- Login CSRF
  - Strict Referer validation
  - Login forms typically submit over HTTPS, not blocked
- HTTPS sites, such as banking sites
  - Use strict Referer validation to protect against CSRF
- Other
  - Use Ruby-on-Rails or other framework that implements secret token method correctly

Web Application Firewalls

- Help prevent some attacks we discuss today:
  - Cross site scripting
  - SQL Injection
  - Form field tampering
  - Cookie poisoning



Sample products:

Imperva
F5 TrafficShield
Citrix NetScaler
CheckPoint Web Intel

Summary

- SQL Injection
  - Bad input checking allows malicious SQL query
  - Known defenses address problem effectively
- XSS – Cross-site scripting
  - Problem stems from echoing untrusted input
  - Difficult to prevent; requires care, testing, tools, …
- CSRF – Cross-site request forgery
  - Forged request leveraging ongoing session
  - Can be prevented (if XSS problems fixed)
- Other server vulnerabilities
  - Increasing knowledge embedded in frameworks, tools, application development